

MeTTaIL for the Working Software Developer

From **SELECT-FROM-WHERE** to **SELECT-FROM-WHERE-DO**,
and what happens when the **WHERE** stops answering yes or no

L.G. Meredith
F1R3FLY.io

August 2026

Abstract

Most treatments of a programming language begin with its data definition language and arrive at control flow later. This one goes the other way round, because the control flow language of rholang 1.4 has a door every working developer has already walked through: the query comprehension. **SELECT-FROM-WHERE** in SQL, *from-where-select* in LINQ, *for-let-where-return* in XQuery — one shape, everywhere, for forty years. LINQ exists because that shape was worth having inside a host language where a developer could finally say what to *do* with the result. Rholang goes the rest of the way and builds the language around the comprehension: `for( ptrn <- chan where cond )P` is **SELECT-FROM-WHERE-DO**, and its dual `chan! (Q)` makes the write side just as explicit as the read side.

Making both sides explicit buys something a query language cannot have. A communication is a meeting between a named reader and a named writer, and the substitution performed by the communication rule is the *witness* that the meeting occurred — so a transaction is a fact about the program text rather than a service of the engine. That raises the question the second half of this note answers: where do the pattern and the condition come from? They come from MeTTaIL's *Theory* block, a data definition language with four rungs — grammar, equations, rewrites — whose top rung defines whole languages, and whose data can wiggle. That structure subsumes SingularityNET's MeTTa: MeTTa's rules are the rewrites and its atom spaces are the channels, but MeTTa has no equational layer and no notion of a witnessed transaction.

Two widenings follow. First, the condition language is not fixed: it is *generated* from the language definition, and the choice of container for witnesses — sets, quantales, join semi-lattices — turns a dial between Boolean, linear and coalition observation, giving a *where* clause different powers of sight. Second, the condition need not be crisp. Graded conditions take values in a resolution algebra, and at that point the same program text stops being a state transition inside a node and becomes a simulation of a population of programs — stochastic over the non-negative reals, quantum over the complex numbers. A developer moves between programming in the small and programming in the large without changing languages, and without maintaining a model separate from the system.

A single running example, a foraging learner drawn from the mortal scientist programme, carries the exposition from the first **SELECT** to the last simulation.

Contents

1	Before anything else	3
1.1	Who this is for	3
1.2	Why the order is backwards	3

1.3	The running example	3
1.4	Notation, and two conventions worth flagging	3
2	The control flow language	4
2.1	The forager, as a query	4
2.2	What LINQ was actually for	4
2.3	SELECT-FROM-WHERE-DO	5
2.4	The other half: the write	5
2.5	The translation table	6
2.6	Joins, groups, and where the <code>where</code> goes	6
2.7	The witnessed transaction	7
2.8	The forager, in full	8
3	Where the pattern and the condition come from	8
3.1	The question the comprehension cannot answer	8
3.2	A <code>Theory</code> block, whole	9
3.3	The four rungs	10
3.4	Rung two: equations, or the comment your type system cannot read	10
3.5	Rung three: rewrites, or data that wiggles	11
3.6	Reuse: the fourth rung	11
3.7	What sharing means, and why the compiler argues about it	12
3.8	Channels are typed by language definitions	13
3.9	MeTTa, and what this adds to it	13
3.10	What an operation costs	14
3.11	What actually runs	15
4	Widening one: what a condition can see	15
4.1	The condition language is generated, not designed	15
4.2	Structural connectives: pattern matching that can talk about namespaces	16
4.3	Behavioural connectives: asking about the future	17
4.4	The collection component is a dial	17
4.5	What the dial costs	18
4.6	The forager's ladder	18
5	Widening two: when the condition stops answering yes or no	19
5.1	The move	19
5.2	What a value algebra has to supply	19
5.3	The guard becomes a weight	20
5.4	Three readings of one program	20
5.5	Programming in the small, programming in the large	21
5.6	The forager, graded	21
5.7	What is settled and what is not	23
6	Where this leaves a developer	23
6.1	The through-line in one page	23
6.2	What to do on Monday	24
6.3	Where to read next	24
6.4	A closing observation	25

1 Before anything else

1.1 Who this is for

This note is written for someone who ships software. It assumes fluency with SQL, comfort with at least one of LINQ, XQuery, list comprehensions or a collection pipeline API, and the ordinary working knowledge of concurrency that comes from having debugged it. It assumes no category theory, no process algebra, and no prior contact with the rho calculus.

There is a companion document, *Graph-Structured Lambda Theories: A Reference for the Working Developer* [30], which covers overlapping ground for a reader who wants the theory first and is willing to spend a chapter on bisimulation to get it. This note is the other entrance to the same building. A reader who finishes here and wants proofs will find them there; a reader who starts there and wants to write code will find that here. Neither is a prerequisite for the other.

1.2 Why the order is backwards

The convention is to teach a language’s data definition language first — here are the types, here is how to make values — and its control flow language second. The convention is defensible and this note ignores it.

The reason is that the control flow language of rholang has a familiar door and the data definition language does not. Every working developer has written a query comprehension. Walking through that door and following the shape until it breaks generates, in order, exactly the questions the data definition language answers. Taught the other way round, the `THEORY` block looks like an unusually elaborate schema declaration and its point is invisible.

So the order here is: the comprehension; what a comprehension cannot do; what rholang adds; what that addition forces us to have; and then, twice, what happens when we widen the thing that was added.

1.3 The running example

One example runs from beginning to end. A *forager* moves through an environment of specimens. Some specimens are rich and worth the cost of opening; some are decoys and are not. The forager has a cheap structural test it can apply, and that test is evidence rather than proof. It has a finite budget, it spends from that budget to look and to open, and if the budget reaches zero the forager stops — permanently.

The example is drawn from a research programme on learning as computation, in which such a forager is a very small instance of a *mortal scientist* [38]: a computation that forms hypotheses about its environment, pays to test them, and is punished by the environment when they are wrong. The point of choosing it is not the biology. It is that the same four lines of code are legible at every stage of this note — as a query, as a transaction, as a hypothesis, and finally as a learning rule — and that the numbers quoted in the last part were measured by running it.

1.4 Notation, and two conventions worth flagging

The calculus underneath rholang is the rho calculus [43]. The name abbreviates **r**eflective **h**igher **o**rders **c**alculus, and dropping the periods gives the transliteration of the Greek letter after π — a joke about what comes after the π -calculus. It is written “rho calculus”, never with the Greek letter.

Quotation is written `@P`: the name that stands for the process `P`. Dereference, sometimes called dropping or unquoting, is written `*x`: the process that the name `x` stands for. These are inverse

in one direction on the nose — @*x is x — and that fact does more work later than it looks like it should.

Code listings in this note come from two places, and it is worth knowing which is which. Listings labelled as running examples are taken verbatim from the `flr3node-rust` repository [46], where they compile and are covered by tests. Listings that illustrate a design under development are marked as such. Nothing here silently pretends that a plan is a product; §3.11 is a table of what actually runs.

2 The control flow language

2.1 The forager, as a query

Start with the forager’s problem, stated the way a developer would state it first. There is a table of specimens. There is a cheap test, χ . The question is which specimens pass the test.

```
SELECT spec
FROM prey
WHERE chi(spec)
```

Three clauses. `FROM` says where the data lives. `WHERE` says which of it we want. `SELECT` says what shape to hand back. The shape is so useful that it has been reinvented in every decade since it was introduced [6, 5].

Dialect	The same query
SQL	<code>SELECT spec FROM prey WHERE chi(spec)</code>
LINQ	<code>from spec in prey where Chi(spec) select spec</code>
XQuery (FLWOR)	<code>for \$spec in \$prey where chi(\$spec) return \$spec</code>
Python	<code>[spec for spec in prey if chi(spec)]</code>
Haskell	<code>[spec spec <- prey, chi spec]</code>

The four descendants are LINQ [14], XQuery’s FLWOR [26], and the list comprehension in the form Wadler gave it [24]. The persistence of the shape across five languages with nothing else in common is the fact worth taking seriously. It is not a syntax fashion. The comprehension is what data access looks like when you write down honestly what you are doing: naming a source, filtering it, and projecting the survivors.

2.2 What LINQ was actually for

There is a step in this history that is usually told as a story about convenience and is better told as a story about expressiveness.

SQL is a complete language for asking. It is not a language for acting. A query returns a result set to *someone else* — an application, in another language, which then does whatever is to be done. The boundary between the two is where the impedance mismatch lives, and every ORM ever written is an attempt to plaster over it.

LINQ exists for precisely this reason: it embeds the query comprehension *into* a host language, so that a developer can at last express the `DO`. That is the whole point. Once `from-where-select` is a first-class expression of C# or F#, the result of a query is an ordinary value in a language that can act on it, and the mismatch is gone — for reads. XQuery’s FLWOR has the same ambition inside XML, and the same limit; XQuery Update [27] was bolted on afterwards precisely because the `return` clause cannot cause anything.

Rholang takes the desideratum LINQ identified and goes the rest of the way. Rather than embedding the comprehension in a host language, it builds the language *around* the comprehension. The comprehension is not a query expression that yields a value; it is the sole way anything happens at all.

2.3 SELECT-FROM-WHERE-DO

Here is the forager’s query again, in rholang.

```
| for( @spec <- prey where Chi(spec) ) {
|   Break!(spec)
| }
```

Clause by clause, against the SQL:

SQL	rholang	What changed
FROM prey	<- prey	the source is a <i>channel</i> , not a table
WHERE chi(spec)	where Chi(spec)	nothing yet; §4 widens it
SELECT spec	@spec	the pattern <i>is</i> the projection
—	{ Break!(spec) }	the DO

Two of those rows deserve more than a line.

The pattern is the projection, and it is stronger than one. A `SELECT` list picks columns. A rholang pattern matches structure: it can destructure nested terms, bind sub-terms to variables, and — as §3 explains — it matches *up to the equations of the theory that types the channel*, so two spellings of the same value match the same pattern without the developer arranging it. A pattern is a projection in the same sense that a regular expression is a substring test.

The DO is a process, not a value. The continuation `{ Break!(spec) }` is not returned to a caller. It runs. It may send on channels, spawn further comprehensions, or do nothing. There is no host language to hand results back to, because there is no outside.

2.4 The other half: the write

A query language has one privileged verb. Rholang has two, and the second is the reason the first behaves differently than it does in SQL:

```
| prey!( spec )
```

That is the write. It is a term, it composes in parallel with everything else using `|`, and — this is the part with consequences — it is *addressed the same way the read is*. The reader names `prey`; the writer names `prey`; neither names the other.

Being explicit about both halves is what distinguishes this from a query language with side effects bolted on. In SQL, `INSERT` and `SELECT` are different kinds of statement, mediated by an engine that owns the table. Here they are two terms in one program, and the table is not owned by anybody.

2.5 The translation table

The following is probably the most useful page of this note for a reader coming from SQL. Rholang distinguishes three ways of reading, and the distinction is exactly the distinction a database developer already makes between a destructive read, a standing query, and a plain look.

rholang	SQL reading
$x! (Q)$	INSERT — put a row on x
$\text{for}(p \leftarrow x)P$	SELECT ... FOR UPDATE, then DELETE, then act. The datum is consumed by exactly one reader.
$\text{for}(p \leq x)P$	a standing query, or a trigger: the reader persists and fires again on the next matching write.
$\text{for}(p \leftarrow x)P$	plain SELECT — a peek. Reads without consuming, so the datum remains for others.
where cond	WHERE
&	a join — and simultaneously a distributed atomic commit across the joined channels.
;	sequence: this group of receipts, then that one.
the pattern	the projection, structural rather than columnar, and matching up to the theory's equations.

The three arrows are worth committing to memory, because choosing among them is the main design decision in a rholang program and the choice is invisible in the SQL vocabulary that motivates it. A linear receipt $\leftarrow$ is the default. It consumes. Persistence $\leq$ and peeking $\leftarrow$ are the deviations, and each gives something up: a persistent receipt gives up the guarantee that a datum is handled once; a peek gives up the guarantee that reading it means nobody else will.

Remark 2.1 (Perception must not be able to act). The difference between $\leftarrow$ and $\leftarrow$ is not efficiency. It is whether looking changes the world. A forager that reads the environment with $\leftarrow$ consumes what it observes; one that reads with $\leftarrow$ observes without disturbing. In the running example this matters immediately: the forager's cheap test χ must be a peek, or the act of assessing a specimen would itself claim the specimen. Getting this wrong produces a system that cannot observe anything without changing it, which is a real bug and not a philosophical one.

2.6 Joins, groups, and where the where goes

The full form of the comprehension is a sequence of groups, and each group is a set of receipts that must occur *together*:

```
for( ptrn_11 <- chan_11 & ... & ptrn_1n <- chan_1n where cond_1 ;
    ... ;
    ptrn_k1 <- chan_k1 & ... & ptrn_km <- chan_km where cond_k ) P
```

Inside a group, & means simultaneity: all of those receipts happen at once or none of them do. Between groups, ; means sequence. Each group carries its own where, and cond_j may mention any variable bound by a pattern in group i for $i \leq j$ — so the guard on a later group can test what an earlier group received. The comprehension commits only when every pattern matches and every condition holds.

A SQL developer will recognise & as a join and should immediately notice the difference. A SQL join is over static extents: the rows are all already there, and the join is a computation over them. A rholang join is over *concurrent writers*: the rows are being produced by other running

programs, and the join is a rendezvous. It succeeds when a matching datum is available on every joined channel at once, and until then it waits.

Aside 2.2 (A note for readers of the papers). Several of the research notes in this line write a single trailing `where` at the end of the whole comprehension. That is an abbreviation. The implemented form is the per-group one above, and it is the better one to learn from, because it makes visible exactly which bindings are in scope for which condition.

2.7 The witnessed transaction

Now the payoff for having made both halves explicit.

When a reader and a writer meet on a channel, one rule fires. In the MeTTaIL presentation of the calculus [45] it is written like this — and this is a real excerpt from a language definition, not pseudocode:

```
| (PPar {(PInput n ^x.p), (POutput n q), ...rest})
  ~> (PPar {(subst ^x.p (NQuote q)), ...rest});
```

In words: a parallel composition containing a receipt on `n` and a send on `n` rewrites to the same composition with both of them replaced by the receipt's body, with the sent term substituted for the bound variable. Everything else, `...rest`, is untouched.

The substitution is the point. After the step, the running term *contains the evidence* that this reader met this writer over this datum: the datum is sitting in the continuation, in the position the pattern named. Nothing external recorded the event. The event left a mark on the program.

Claim 2.3 (Transactions are a property of the term). *A communication is atomic, and its atomicity requires no support from the runtime. The condition is evaluated before the step commits; if it fails, nothing is consumed, so there is no partial state and nothing to compensate. If it succeeds, exactly one reader and one writer are consumed and the substitution records that they were the ones. There is no state in which the read happened and the write did not.*

This is not a subtle argument, and the implementation makes it concrete. The comment at the head of the shipped example [46] says it plainly: a failing guard leaves the message in the tuple space, exactly as a non-matching pattern does. A failed transaction is not rolled back; it never began.

```
// Receive-side 'where' guard: only commit when both the spatial pattern
// matches AND the guard evaluates to true. Failing the guard leaves the
// message in the tuple space -- same as a non-matching pattern.
new chan, stdout(`rho:io:stdout`) in {
  chan!(-3) |
  chan!(0) |
  chan!(7) |
  for (@x <- chan where x > 0) {
    stdout!(`kept`, x)
  }
}
```

Three consequences a developer will care about.

Multi-party commit is a language construct. A join & across m channels is one transaction over m resources. It either takes a datum from each or takes none. The protocol machinery that would normally implement this — prepare, vote, commit, the coordinator, the timeout — is not written, because the rule that fires is the multi-channel one.

The witness is where cost attaches. Because there is an identified event, there is something to charge for. This is what makes the metering of §3.10 possible without instrumenting an interpreter: the unit of billing is the same unit as the unit of commitment.

Candidates. For a join across several channels with several available data, there may be more than one way to assign data to patterns. Each such assignment is a *candidate*. Ordinarily one of them happens and the others do not, and a developer never thinks about it. Keep the word anyway: candidates are the thread that runs through the rest of this note. In §4 a condition is evaluated per candidate; in §5 a candidate carries a weight rather than a verdict, and everything interesting follows from that.

2.8 The forager, in full

Enough machinery is now on the table to write the running example properly. The forager has a belief, held as data on a channel. It races two courses of action: open the specimen, or ignore it and graze a low-yield but safe alternative.

```
new bel, prey, wall in {
  bel!( (1, 0.5) ) | // (n, s): one unit of prior evidence

  // the two branches race for the belief and the specimen
  for( @(n,s) <- bel ; @spec <- prey where Chi(spec) ) {
    Break!(spec) | bel!( revise(n, s, spec) )
  } |
  for( @(n,s) <- bel ; @spec <- prey where g ) {
    Graze!(*wall) | bel!( (n, s) )
  }
}
```

Read it as a developer. Two comprehensions compete for the same two resources: the belief and the next specimen. Whichever fires consumes both, does its work, and puts an updated belief back. Because the belief is consumed, the two branches cannot both act on the same specimen — mutual exclusion is not arranged, it is what a linear receipt means.

Three questions are now unavoidable, and they are the agenda for the rest of this note.

1. `Chi(spec)` is a condition about the *structure* of a specimen. Where does the vocabulary for saying that come from? (§3.)
2. What else could a condition say, beyond structure — and what does saying more cost? (§4.)
3. `g` in the second branch is not a Boolean. It is a number. What is a numeric `where` clause, and what does a program mean when its conditions stop answering yes or no? (§5.)

3 Where the pattern and the condition come from

3.1 The question the comprehension cannot answer

The comprehension of §2 presupposes two things it does not supply. Its pattern `@spec` must have a grammar of terms to match against. Its condition `Chi(spec)` is a claim about structure, so there must be a notion of what structure a specimen has.

In SQL both come from the schema. `spec` is a row of a known table with known column types, and `chi` is a predicate over those columns. The schema is the data definition language, and the query language leans on it entirely.

Rholang's schema is the `Theory` block, and the reason to keep reading is that it is a great deal more than a schema.

3.2 A Theory block, whole

Here is one, complete [45]. It defines the rho calculus [43] itself — which is a useful thing to see early, because it shows that the calculus rholang is built on is not privileged: it is one language definition among others, expressible in the same block a developer would use for an invoice format.

```
Theory RhoCalc() {
  Types {
    Proc;
    Name;
  }
  Terms {
    PZero . |- "0" : Proc;
    PDrop . n:Name |- "*" "(" n ")" : Proc;
    POutput . n:Name, q:Proc |- n "!" "(" q ")" : Proc;
    PInput . n:Name, ^x.p:[Name -> Proc] |- n "?" x "." "{" p "}" : Proc;
    PPar . ps:HashBag(Proc) |- "{" ps.*sep("|") "}" : Proc;
    NQuote . p:Proc |- "@" "(" p ")" : Name;
  }
  Equations {
    (NQuote (PDrop N)) == N;
  }
  Rewrites {
    RComm : (PPar {(PInput n ^x.p), (POutput n q), ...rest})
      ~> (PPar {(subst ^x.p (NQuote q)), ...rest});
    RDrop : (PDrop (NQuote P)) ~> P;
    RPar : if S ~> T then (PPar {S, ...rest}) ~> (PPar {T, ...rest});
  }
}
```

Four blocks. `Types` names the syntactic categories. `Terms` gives one constructor per line: a label, then the arguments it takes with their sorts, then — after the turnstile — the concrete syntax written in terms of those argument names, then the category it produces. So the grammar and the pretty-printer come from the same declaration. `Equations` says which distinct terms are to be regarded as the same. `Rewrites` says how terms move — in the style Plotkin introduced for operational semantics [16] and Milner used to such effect for processes [15].

From that block, the toolchain generates the abstract syntax types, a parser and a bidirectional printer, capture-avoiding substitution that respects binding, and a rewrite engine.

Three details in the `Terms` block repay a second look, because each is carrying weight a conventional schema language has no way to carry. `^x.p:[Name -> Proc]` declares that `PInput`'s second argument is an *abstraction*: `x` binds in `p`. Binding is part of the data declaration, which is why substitution can be generated rather than hand-written. `ps:HashBag(Proc)` declares that `PPar` takes an unordered collection with multiplicity, and `ps.*sep("|")` says how to render one. And in `RComm`, `...rest` binds whatever else was in the bag and puts it back — which is what lets a rule fire on two participants inside an arbitrary parallel composition without saying anything about the others.

3.3 The four rungs

The blocks form a ladder, and where a given language definition stops on that ladder determines what kind of thing it is.

Blocks populated	What you can present	The familiar name
Types, Terms	free term algebras	algebraic data types
+ Equations	quotients of free algebras	finitely presentable universal algebra
+ Rewrites	theories with dynamics	domain-specific languages
+ parameters and the theory algebra	theories built from other theories	modules, in the sense of ML functors

An ordinary data definition language stops on the first rung. Records, sums, products, lists, maps: that is the whole of what most type systems can say about data, and it is a lot. The second and third rungs are where this differs, and the second rung is the one that will surprise a working developer most, so take it first. The fourth is the subject of §3.6, and answers the question every developer asks on seeing the block above: *can I reuse part of one of these?*

3.4 Rung two: equations, or the comment your type system cannot read

Every codebase of any size contains comments of this form:

This is a list, but order doesn't matter — always compare as a set.
This is a string, but compare case-insensitively.
Addresses with an empty line-2 are equal to the same address without it.

Each is an equation. Each is being maintained by hand, in prose, because the type system cannot express it. The consequences are familiar: a hand-written `equals`, a normalisation function called on the way in and forgotten in one code path, a cache keyed by the wrong representative, and a class of bug that only appears when two spellings of the same value meet.

An `equations` block moves that comment into the definition:

```
Equations {  
  (NQuote (PDrop N)) == N;  
}
```

The payoff is not that the compiler now checks something. It is that *pattern matching happens up to the equations*. When the comprehension of §2 matches `@spec` against a datum, it matches modulo the theory that types the channel. A developer never writes normalisation, never chooses a canonical form, and never has the bug where one code path forgot.

This is why the `PPar` constructor above takes a `HashBag` rather than a list: parallel composition is associative and commutative, and rather than being a special case hard-wired into an interpreter, that is a property of the data declaration. The same mechanism is available for an invoice.

Aside 3.1 (The reach of the second rung). “Finitely presentable universal algebra” is the mathematician’s name for the second rung, and it is worth translating: it means anything you can specify by saying what the constructors are and which combinations of them are equal. Monoids, groups, lattices, sets-as-quotients-of-lists, normalised paths, case-insensitive strings, and commutative parallel composition are all on this rung. Most of a data model’s invisible invariants live here.

3.5 Rung three: rewrites, or data that wiggles

The third rung adds `rewrites`, and with it the thing an ordinary schema cannot do: the data moves on its own.

```
Rewrites {  
  Beta : (App (Lam ^x.body) arg) ~> (subst ^x.body arg);  
  AppCongL : if M0 ~> M1 then (App M0 N) ~> (App M1 N);  
}
```

That is the lambda calculus, in two of the four lines a real definition takes. The first line is the computation rule — and note that beta reduction is literally the substitution of an argument into an abstraction, written as such. The second is a congruence rule saying where the first is allowed to fire; the `if . . . then` premise is the horizontal bar of an inference rule. Together they make a term of type `Term` not an inert value but a small running program.

Once a channel is typed by such a definition, sending on that channel means sending a term of a language — a term that can be inspected, matched against, and stepped by the recipient. Two agents that share a language definition can exchange programs in a domain-specific language constructed for the purpose, and the recipient knows what it received because the definition came with it.

The slogan for this rung is *the data can wiggle*, and its practical form is that the boundary between data and code, which most systems maintain by convention and defend with a serialisation format, is a dial here rather than a wall.

3.6 Reuse: the fourth rung

A developer who has just written the `RhoCalc` block above will notice something about it. Parallel composition is associative, it is commutative, and `PZero` is its unit. That is a commutative monoid. It is also, verbatim, the structure of integer addition, of string concatenation up to reordering, of set union, and of every other commutative monoid the developer has ever implemented. Writing it out again is the same waste it always was.

The fourth rung is the answer, and it is the familiar one: a language definition takes parameters.

```
Theory EmptySet() {  
  Types { Elem; }  
  Exports { Elem; }  
}  
  
Theory Monoid(s: EmptySet) {  
  s  
  Terms {  
    One . |- "1" : Elem;  
    Mult . x:Elem, y:Elem |- "(" x "*" y ")" : Elem;  
  }  
  Equations {  
    (Mult (Mult x y) z) == (Mult x (Mult y z));  
    (Mult x (One)) == x;  
    (Mult (One) x) == x;  
  }  
}  
  
Theory CommutativeMonoid(m: Monoid) {  
  m
```

```

Replacements {
  One => Zero . |- "0" : Elem;
  Mult => Plus . x:Elem, y:Elem |- "(" x "+" y ")" : Elem;
}
Equations {
  (Plus x y) == (Plus y x);
}
}

```

Two things are happening. `Monoid` takes a theory as a parameter and extends it: this is an ML functor, or a generic class, or whatever the reader's language calls a module parameterised by another module. And `CommutativeMonoid` does something a generic class cannot: a `Replacements` block *renames an inherited constructor and gives it new concrete syntax*. `Zero` and `Plus` are not new operations sitting alongside `One` and `Mult`; they *are* `One` and `Mult`, spelled additively. Every equation proved of the one holds of the other, because they are the same element of the same theory.

Which is what lets `rholang`'s parallel composition be defined rather than declared:

```

import "UnivAlg.module" as u

Theory ParMonoid(cm: u.CommutativeMonoid) {
  cm
  Exports {
    Elem => Proc;
  }
  Replacements {
    Zero => PZero . |- "0" : Proc;
    Plus => PPar . ps:HashBag(Proc) |- "{" ps.*sep("|") "}" : Proc;
  }
}

```

Parallel composition is commutative-monoid multiplication with the carrier renamed to `Proc`. That is the whole definition. Associativity, commutativity, and the unit law arrive with it and are not restated.

Aside 3.2 (Why the bag, again). Notice what `PPar` did to arity on its way through. `Plus` is binary. `PPar` takes a `HashBag`. A bag is precisely a sequence quotiented by associativity and commutativity, so moving to a bag turns two of the inherited equations from things checked during matching into things true of the representation. This is the second rung and the first rung meeting: an equation, made structural.

3.7 What sharing means, and why the compiler argues about it

Theories compose with three operators — $\vee$ for combining, $\wedge$ for the common fragment, $\setminus$ for subtraction — and the first of these has a subtlety that is worth a developer's attention, because the compiler will raise it whether or not the developer was expecting the conversation.

Consider building the rho calculus in pieces: one theory adds restriction and replication, another adds send and receive, and both are built on `ParMonoid`. Combining them must not produce two parallel compositions.

```

Theory Rholang(nr: NewReplCalc, rc: RhoCalc) {
  nr \ / rc
}

Theory FreeRholang() {

```

```

let s = u.EmptySet() in (
let m = u.Monoid(s) in (
let cm = u.CommutativeMonoid(m) in (
let pm = ParMonoid(cm) in (
let qd = QuoteDropCalc(pm) in (
let nr = NewReplCalc(qd) in (
let rc = RhoCalc(qd) in (
  Rholang(nr, rc)
))))))
}

```

The `let` chain is doing the work. `QuoteDropCalc(pm)` is elaborated once and bound to `qd`; both `nr` and `rc` are built from that one instance; so when they are combined, everything they inherited from it is recognised as the same thing arriving by two routes, and appears once. The combination is a pushout over the shared ancestor rather than a union.

The practical consequence is that *sharing is a claim, and the compiler holds you to it*. Build the two halves from two separate instances and combine them, and the result is a diagnostic rather than a silently doubled calculus:

```

[join-collision] cannot join: label 'PPar' was
introduced independently on both sides

```

This is unfamiliar and it is worth sitting with, because it is the same diagnostic that catches genuine modelling errors. A rig has one carrier and two monoid structures on it — addition and multiplication — and the natural way to write it is to instantiate `Monoid` once and pass it as both arguments. That is wrong: it asserts that addition and multiplication are the same operation. The compiler says so. Instantiating `Monoid` twice over one shared `EmptySet` is the correct structure, and it says exactly what a rig is: two monoids, one carrier.

A developer used to inheritance will recognise the diamond problem here, and should notice that the resolution is different in kind. There is no linearised method resolution order and no “last one wins”. Two elements are the same element if they have the same origin, and otherwise they are two elements; if they also have the same name, that is an error rather than a silent override.

3.8 Channels are typed by language definitions

The connection back to §2 is one sentence: *a channel’s type is a language definition*.

That single design decision explains the rest of the note. Sending on the channel means sending a term of that theory. Matching a pattern on it means matching a term of that theory, up to that theory’s equations. And — the point of §4 — asking a condition about what arrived means asking a question in a logic that is *generated from* that theory, because the theory says what there is to ask about.

For the forager, this means the specimen language is a language definition, `Chi` is a formula in the logic that definition generates, and there was never a moment at which someone hand-designed a predicate vocabulary.

3.9 MeTTa, and what this adds to it

Readers coming from SingularityNET’s Hyperon stack [11] will have recognised the shape of the third rung, because MeTTa is built on it. The relationship is worth stating precisely, in both directions.

MeTTa	MeTTaLL	Relationship
rules	the <code>rewrites</code> block	subsumed; MeTTa’s rules are rewrites in this sense
atom spaces	channels	subsumed, and typed: a channel carries terms of a declared language, an atom space carries atoms
—	the <code>equations</code> block	absent in MeTTa: no equational layer, so every quotient must be hand-coded as rules, and confluence becomes the programmer’s problem
query, then update	one witnessed communication	absent in MeTTa: query and update are separate acts, so there is no atomic multi-party meeting and no witness

The two gaps compound. Without equations, a developer who wants “these two atoms are the same” writes rewrite rules that say so, and then owns the confluence question — whether the order in which rules fire can change the answer. Without a witnessed transaction, a developer who wants two atom spaces updated together writes a protocol, and then owns the partial-failure question. Both are exactly the questions the two blocks and the communication rule answer for free.

None of which is a dismissal. MeTTa has a large working ecosystem, a substantial body of practice around the atom space as a shared knowledge medium, and Probabilistic Logic Networks [12], which is a serious piece of engineering for reasoning under uncertainty. It is worth saying plainly that PLN is not left behind here: in §5 it comes back as one of the value algebras a `where` clause can be graded in, which is to say that PLN inference can drive execution rather than sitting beside it. The relationship this note is proposing is embedding with an addition, not replacement.

3.10 What an operation costs

One more piece belongs to the data definition story, because it is the thing that makes the forager’s budget real rather than notional.

Cost accounting in this system adds exactly two surface forms, and both of them lower to ordinary rholang:

Surface	Lowers to	Which is
<code>s :: ()</code>	$\Sigma[s]!(\text{Nil})$	a send: mint a token
<code>{% P %}[s]</code>	<code>for(t <- $\Sigma[s]$) { *t P }</code>	a receipt: run P once a token arrives

That is the entire mechanic. Spending fuel *is* a communication: the gate consumes one token, releases its payload, and then P runs. If no token is available on the signature channel, the gate parks forever and P never runs — and that is what metering means here. A budget is a stack of nested sends, and its depth is how many operations it funds.

Two things follow that a developer should notice.

First, metering is not a virtual-machine feature bolted onto a language. It is a program *in* the language, and the transpiler’s job is only to write it for you. Anything true of ordinary rholang is true of metered rholang.

Second, it composes with Claim 2.3 in a satisfying way. A communication that is not funded is a communication that does not happen, and it is indistinguishable from a communication whose guard failed. Both are non-events. There is no separate failure mode for running out of budget,

no exception, and nothing to unwind — which is exactly the property that lets the forager’s death be modelled as “it stops” rather than as an error path.

The repository [46] ships a worked demonstration of this at scale: a supply chain in which factories produce, carriers ship, sellers wholesale and buyers retail, with money and inventory both conserved, and every single operation gated by a token. It is the best available answer to “show me this doing something real”.

3.11 What actually runs

Honesty about status is more useful to a developer than enthusiasm, so the table below separates three things a reader has every right to keep apart: *specified* means the surface is frozen and written down; *implemented* means there is code a reader can run; *planned* means neither yet.

Capability	Status
Theory blocks: types, terms, equations, rewrites; generated parser, printer, substitution, rewrite engine	implemented, in the reference elaborator and in <code>mettail-rust</code> [45], with language definitions shipped for lambda, ambient, the rho calculus and a calculator
The fourth rung (§3.6): theory parameters, <code>Exports</code> , <code>Replacements</code> , the theory algebra, and pushout sharing (§3.7)	specified and implemented in the elaborator, with the universal-algebra tower and the rho calculus as its test corpus; <i>not yet</i> in the <code>flr3node-rust</code> parser, which is the next step
Imports by URL	specified; resolution is a compile-time step in the client toolchain, so a node never performs it
<code>where</code> guards on receipts, and <code>where</code> guards on <code>match</code> cases with fall-through	implemented on the <code>cost-accounting-transpiler</code> branch of <code>flr3node-rust</code> [46], with runnable examples and compile tests
Cost accounting: minting, gates, compound signatures, budget stacks	implemented on the same branch, with a transpiler and a documented demo
Conditions drawn from the <i>generated logic</i> (§4) rather than from ordinary boolean expressions	planned. The guard slot exists and is evaluated before commit, but a condition today is an ordinary boolean expression; the structural and behavioural connectives are specified in the research notes, and connecting an elaborated theory to the guard is what would close the gap
Graded conditions and simulation (§5)	implemented as a separate simulator crate [41], with the stochastic and quantum readings both built and tested; not part of the node’s execution path, by design (§5.2)

The condition language a developer can write *today* is boolean expressions over the variables the patterns bound — $x > 0$, and so on. Everything in the next section is about what that slot is going to hold, and why the choice is more interesting than it looks.

4 Widening one: what a condition can see

4.1 The condition language is generated, not designed

Here is the claim of this section, stated first so the rest can be read as support for it.

The vocabulary available in a `where` clause is not a fixed list of built-in predicates, and nobody sat down and designed it. It is *constructed from the language definition* that types the channel. Change the language definition, and the condition language changes with it, automatically and in a specified way.

That is worth dwelling on because the alternative — a fixed predicate library — is what every other system does, and its failure mode is familiar. A query language ships with the predicates its designers anticipated. When a developer needs one that was not anticipated, the escape hatch is a user-defined function, which is opaque to the optimiser, the index, and the type system. Generated conditions have no escape hatch because they have no fixed list to escape from.

A generated condition language has three components, and the account of why it has exactly these three is a realizability argument set out elsewhere [31]. Two of them come from the language definition and one is a choice.

Component	Where it comes from	What it lets you say
structural	one connective per term constructor in the <code>terms</code> block	“this term is a send on a channel satisfying ...”, “this splits into two parts sharing nothing”
behavioural	one modality per rule in the <code>rewrites</code> block, and per position it could fire at	“after one step of <i>this</i> rule, ... holds”
collection	a choice : what container holds the witnesses	which logical connectives exist at all, and how they behave

4.2 Structural connectives: pattern matching that can talk about namespaces

The structural component is the easiest to get a feel for, because it is recognisably a generalisation of pattern matching.

Every constructor in the `terms` block yields a connective. If the theory has `POutput . n:Name, q:Proc |- n "!" "(" q ")" : Proc`, the logic gets a way to say “a send, on a name satisfying φ , of a term satisfying ψ ”. Nested, these compose into descriptions of arbitrary shape.

Two of the generated connectives are worth naming because they do things ordinary patterns cannot.

Separating conjunction. Because parallel composition is associative-commutative — an equation, from rung two — the connective it generates is not ordinary conjunction but a *separating* one: $\varphi \mid \psi$ holds of a term that splits into two parts, one satisfying φ and one satisfying ψ , *with the split being a real partition*. This is the same idea as separation logic’s separating conjunction [19], and the same connective the spatial logics for concurrency carry [4], and it is what lets a condition say “there are two of these, and they are not the same one” — a sentence ordinary conjunction cannot express.

The name predicate. Because names are quoted terms, a condition can descend from a name into the term it quotes [44]: $@\varphi$ holds of a name whose quoted process satisfies φ . In developer terms, this means a condition can talk about *namespaces* — about the shape of the addresses involved, not just the payloads. The forager’s belief channel and the specimen channel are distinguishable to a condition by what kind of name they are, which is how one describes a region of the address space rather than enumerating it.

4.3 Behavioural connectives: asking about the future

The behavioural component is the one with no analogue in a query language. For each rewrite rule, and each position in the term where that rule could fire, the logic gets a modality $\langle K \rangle \varphi$ — a refinement of the modality of Hennessy–Milner logic [13], indexed by rewrite rule and position rather than by action [42] — read: *there is a step of this rule at this position, after which φ holds*.

The forager’s use for this is immediate. $\text{Chi}(\text{spec})$ as written is a structural question — does this specimen *look* a certain way. A behavioural question is different: does this specimen, if poked, *do* a certain thing. Both are legitimate hypotheses. They are priced very differently.

Remark 4.1 (Read or drive). There are two ways to learn any property of the environment. *Read* it, if the environment has already computed the property into visible structure — and pay only for looking. Or *drive* it, by making the environment take a step and observing what happens — and pay for the step. The structural connectives price the first, the behavioural connectives price the second, and which is cheaper is a fact about the environment rather than about the observer.

Remark 2.1 now acquires a sharper form. Driving means acting, and acting on the thing you are trying to observe changes it. A system that wants perception to be non-disturbing must arrange that its observational apparatus uses peeks and its interactions use linear receipts — and the two must sit in different namespaces, so that a formula meant to look cannot accidentally claim. In the forager, reading the specimen’s structure and opening the specimen must be different names, or the assessment eats the subject.

4.4 The collection component is a dial

The third component is a choice, and it is the most consequential one in this section. A logic needs somewhere to put its witnesses — the things that make a formula true — and the container chosen determines which logical operations even exist.

Container	Logic you get	What the where can now ask
sets (powerset)	the usual Boolean operations: and, or, not, implication	“does this hold?” — the familiar case, and what the implementation does today
quantales [21, 28]	the linear connectives [10]	“is there <i>exactly one</i> of these, to be consumed once?” — resource questions, where using a witness spends it
join semi-lattices	coalition logic [22, 7]	“does some group of these participants agree?” — quorum questions, asked about a population without running a protocol

The distinction between the first two rows is the distinction between counting and noticing. A Boolean condition, asked whether there are three red items, answers a question about presence: there *are* red ones. It cannot count them, because in a set two witnesses of the same kind collapse into one. A linear condition keeps them apart, because in a quantale they do not collapse. If you have ever wanted a query that says “at least three distinct matching rows, and consuming one of them consumes it for everyone”, the second row is where that lives.

The third row is stranger and more useful than it sounds. Coalition logic asks about groups: is there a set of participants such that, together, they can bring something about. Read as a condition on a running population — other programs in the environment — it asks whether a quorum has formed [37]. The forager can use this to ask whether the specimens around it are of a kind that behaves as a group, without participating in any protocol they might be running.

Remark 4.2 (Same question, three sights). The reason to present this as a dial rather than three logics is that the question being asked is the same question, and the theory being asked about is the same theory. The structural and behavioural connectives are unchanged. What changes is only what a witness is, and therefore what conjunction and disjunction do to witnesses. This is why the arrangement is not ad hoc: a developer is not choosing among logics somebody designed, but among *observation disciplines* over a structure that was already there [32].

4.5 What the dial costs

Turning the dial up is not free, and the price is worth stating in the terms a developer cares about.

A more expressive condition may not terminate. The behavioural modality asks about steps, and a term can step forever. Fixed points over the modality — “it keeps being able to do this” — are expressive and unbounded. The mitigation is that evaluating a guard is metered like everything else: a budget bounds the check, and an unfunded check is a non-event by §3.10, so it fails safe.

Restricting to make checking terminate loses something real. A developer who wants guards that always answer can restrict to a fragment — fewer connectives, or a smaller slice of the theory. That works. What is surrendered is *adequacy*: the property that the logic can distinguish any two programs that actually behave differently. In general, a logic whose model checking always terminates cannot be adequate for a Turing-complete theory. This is not a defect of the construction; it is the shape of the trade, and the dial exists precisely so it can be made deliberately rather than by accident.

Adequacy is a property of the target, not of the generator. It is tempting to assume that a machine-generated logic comes with a guarantee. It does not, and it should not: whether the generated logic is adequate depends on which connectives were asked for. Without something like a negation and a conjunction, it will not be. A generated *type system*, for instance, is generated by the same family of algorithms and is not expected to be adequate at all [42, 30].

4.6 The forager’s ladder

Return to the running example. `Chi(spec)` was a placeholder for one hypothesis. The dial says there is a whole ladder of possible hypotheses, from cheap structural ones to expensive behavioural ones, and the forager can afford to sit anywhere on it.

That is not a metaphor: it has been measured. In a companion study [39], a learner of exactly this shape was equipped with a ladder of hypotheses expressed as `where` clauses, each priced by the depth of the formula it installs, and run to exhaustion against a fixed environment.

Measurement	What it says
The non-modal rungs delivered 92.3% of the total gain for 34.3% of the total cost	structural sight is the bargain; behavioural sight is the luxury
The single best purchase on the whole ladder was a <i>name-level</i> predicate at modal depth zero	namespaces are cheap to look at and highly informative
The rung that quantified over all replies to all moves was the worst purchase measured	alternating quantification over the future is where cost explodes
A learner holding the <i>complete, correct</i> theory was dominated by a learner holding a cheaper wrong one	truth buys safety, not yield; and the environment, not the learner, decides whether truth is affordable

The last row is the one to carry forward. It says that the right position on the dial is not a property of the problem, and certainly not a matter of rigour. It is an economic question about the environment the program is deployed into, and it is answerable — because both the sight and the acting are priced in the same currency.

5 Widening two: when the condition stops answering yes or no

5.1 The move

A guard returns a truth value, and the communication fires or does not. That is one choice, not the only one. The second widening is to let a condition return a value in some other algebra — a probability, a rate, a cost, a complex amplitude — and to interpret that value as *how much* this candidate happens rather than *whether* it happens.

Before anything else, the reassurance a developer needs:

Proposition 5.1 (Conservativity). *When the value algebra is the Booleans, a graded where clause is exactly the where clause of §2. Nothing about existing programs changes, and no existing program acquires a new meaning.*

So this is an extension in the strict sense. A developer who never grades a condition never encounters any of what follows. A developer who grades one gets something that turns out to be much larger than a weighted choice.

5.2 What a value algebra has to supply

Not every set of values will do. The condition language has conjunction and disjunction in it, so whatever the values are, there must be an operation to interpret each. The requirement is mild: a commutative semiring [33]. Multiplication $\otimes$ interprets conjunction, addition $\oplus$ interprets disjunction, and the units behave as true and false do.

Values	$\otimes, \oplus$	A condition means	Running the program means
$\mathbb{B}$	and, or	a guard	executing, as today
$\mathbb{R}_{\geq 0}$	$\times, +$	a rate or propensity	a stochastic simulation
$[0, 1]$ with PLN values	$\times, +$ on the power <i>sc</i>	a graded belief	inference driving execution
Viterbi $(\max, \times)$	$\max, \times$	the best explanation	finding the most likely path
tropical $(\min, +)$	$\min, +$	a cost	finding the cheapest path
$\mathbb{C}$	$\times, +$	an amplitude	a quantum simulation

Which value algebra a program is read in is not merely a choice of numerics: it fixes how much of the non-determinism a resolver has to decide, and that question has a precise hierarchy behind it [36]. One casualty is worth naming immediately, because it surprises people. Negation does not survive. Over $[0, 1]$ the map $x \mapsto 1 - x$ behaves well enough, but over the complex numbers there is no canonical negation and De Morgan fails. The practical consequence is that the design keeps two sorts of formula: a *crisp* sort with the full Boolean structure, used for gating, and a *graded* sort with a semiring only, used for weighting. A *where* clause keeps its old job and gains a new slot.

5.3 The guard becomes a weight

Recall candidates from §2.7: for a comprehension with data available, each way of assigning data to patterns is a candidate. In the crisp setting the condition is evaluated per candidate and admits or refuses it. In the graded setting the condition is evaluated per candidate and *scores* it.

That is the entire mechanism, and three worthwhile things fall out of it without further work.

The propensity is the formula’s value, and formulas move. A crisp guard is a fixed policy: it fires on the same specimens forever, because the formula does not change. A graded guard’s behaviour is the formula’s *value*, evaluated against the current state — and the current state includes whatever the program has learned. Nothing needs to be scheduled.

Weight tables are unnecessary. A common design for stochastic execution [41] attaches a table of rates to each rule, keyed by conditions, and then requires those conditions to partition the space so that lookup is single-valued. A graded clause needs none of that: a formula has one value, so single-valuedness is free, and overlapping conditions are simply fine.

Plasticity is free. Because the clause is evaluated against the current term, weights that change during execution require no update mechanism. The weight is not stored; it is computed.

5.4 Three readings of one program

Here is where the section earns its title. The same program text, under three different value algebras, is three different kinds of artefact.

Booleans: a state transition. The program is what it has been all along — code running in a node, manipulating state, committing transactions. Programming in the small.

Non-negative reals: a stochastic simulation. Each candidate carries a rate. The total rate at a configuration determines how long the system waits and which step it takes; this is the standard Gillespie construction [9], and it turns the program into a generator of trajectories — the same move that took the π -calculus to the stochastic π -calculus and to SPiM [17, 18], and that the weighted reading of a language definition generalises [41]. Running it many times gives you the distribution of outcomes rather than one outcome. Programming in the large: the subject is no longer this program’s state but the behaviour of a *population* of them.

Complex numbers: a quantum simulation. Each candidate carries an amplitude [34], and candidates that reach the same outcome have their amplitudes added before magnitudes are taken — which is to say, they interfere. Two branches that are individually possible can cancel — the effect Shor’s algorithm is built out of [23]. A developer’s summary: disjunction stops being monotone, and a false disjunction with true disjuncts becomes expressible.

Remark 5.2 (The simulator is not the interpreter). This matters operationally and is easy to get wrong. The graded readings do not run inside the node. They are a separate execution path with its own state, running offline, whose output is a graph or an ensemble rather than a committed state change. That separation is what makes things affordable there that are not affordable in the node — exhaustive construction of the reachable graph, model checking, fixed points — and it is what keeps a consensus-critical execution path free of any of it.

5.5 Programming in the small, programming in the large

The reason to care is not that a simulator exists. Simulators exist. The reason to care is that *it is the same program text*.

The usual arrangement in industry is two artefacts: the system, written in one language by one team, and the model of the system, written in another language by another team, for capacity planning or risk or verification. The model is always a little wrong, it goes stale immediately, and the discrepancy is discovered in production.

Here the model and the system are the same source. A developer writes a comprehension with a graded condition; under the node’s resolver it is a transaction, and under the simulator’s resolver it is a model of what a population of such transactions would do. There is no translation step in which fidelity is lost, because there is no translation step.

The practical questions this makes askable, in the language the system is written in:

- If this contract is deployed a thousand times against that environment, what is the distribution of outcomes?
- Which of these two policies survives longer under a shift in the environment?
- Is there a configuration reachable from here that we would consider a failure, and how much would getting there cost an adversary?

5.6 The forager, graded

Now the running example, all the way through. The forager’s clause becomes:

```
new bel, prey, wall in {
  bel!( (1, 0.5) ) | // (n, s): one unit of prior evidence

  for( @(n,s) <- bel ; @spec <- prey where Chi(spec) * Pow(n,s) ) {
    Break!(spec) | bel!( revise(n, s, spec) )
  } |
  for( @(n,s) <- bel ; @spec <- prey where g ) {
    Graze!(*wall) | bel!( (n, s) )
  }
}
```

$\text{Chi}(\text{spec})$ is the crisp structural predicate of §4, entering the graded sort as an indicator. $\text{Pow}(n, s)$ is a graded ground formula whose value is the PLN power $s \cdot c$, where $c = n/(n + k)$ is

confidence and k is the learner’s scepticism [12]. $*$ is graded conjunction. g is a scalar: the value of grazing.

Two comprehensions, each with a weight. The forager opens the specimen with propensity sc against the wall’s g . Everything below was measured by running exactly this.

The bootstrap trap, and why exploration is a disjunct. A belief with no evidence has $c = 0$, hence power 0, hence a clause value of zero, hence a receipt that never fires — so it never acquires evidence. Measured: a forager initialised with no prior evidence opens 0.00 specimens across 400 encounters and ends with exactly the grazing baseline, against 3543.7 for the same forager given one unit of prior evidence.

The remedy is written in the clause language, not in a framework. Either the belief carries prior evidence $n_0 \geq 1$, or the clause is $\text{Pow}(n, s) \oplus \varepsilon$ for a small scalar ε . That second form is ε -greedy exploration, and it is a *disjunct*.

Confidence buys variance, not yield. Four foragers, two worlds, 20,000 lives each. In the benign world the crisp forager — which acts on a satisfied guard from its first encounter — has the *highest* mean terminal holdings, 4084.7, and the *lowest* survival, 0.8501. The graded forager has survival 0.9485 and holdings 3543.7. That is the trade a mortal program actually faces, and a Boolean guard cannot express it.

A Boolean guard has no coordinate in which to record disappointment. In the hostile world, where opening is a net loss, the crisp forager survives 0.29% of the time. It cannot do otherwise: its policy *is* the formula, and the formula does not change. The graded forager’s policy is the formula’s value, which moves:

opening	s	c	power	$\text{Pr}[\text{open}]$
1	0.7500	0.3333	0.2500	0.3333
2	0.5000	0.4286	0.2143	0.3000
3	0.3750	0.5000	0.1875	0.2727
5	0.2500	0.6000	0.1500	0.2308

Strength falls as decoys accumulate, confidence rises, the product falls, and with it the willingness to open. The forager learns to stop. Note that nothing in the clause is a schedule: early caution and late decisiveness are consequences of a coordinate of the truth value, not of a hyperparameter someone tuned.

Your where clause has become a learning rule. This is the point at which the exposition arrives somewhere a developer would not have predicted from §2. In a one-dimensional grading, the revision step that updates the belief turns out to be a *natural gradient* step [2] on the underlying likelihood — to machine precision, verified over 20,000 random instances at a worst discrepancy of 1.1×10^{-16} . Better, it is an *implicit* step in the sense of stochastic approximation [20], and therefore unconditionally stable: it cannot leave the valid range of values for any observation, where the explicit version can. The confidence coordinate is not a second belief. It is the metric with respect to which the first one is being learned.

The score that the gradient is assembled from decomposes into contributions computable at each communication site, from what the site already has — which is the identity that makes policy-gradient methods implementable [25] and, here, makes the gradient a message rather than a backward pass. The developer’s version of that sentence: grading a condition gives you gradient

descent in the small, correctly conditioned, without importing a learning framework and without a separate training loop. The learning rule is the clause.

5.7 What is settled and what is not

Three honest boundaries, since the previous subsection was enthusiastic.

The stochastic reading is solid; the quantum reading has a fence around it. The complex-valued case is established for a restricted fragment — finite, contractive, staged — and not for the language as a whole. Two specific hazards sit behind that fence. Left unrestricted, the construction buys post-selection, and a language with free post-selection decides more than quantum computers do [1]. And the coherent branching this needs is quantum *control* rather than the architecture of the quantum process calculi, which keep the quantum state beside the process rather than in the branching [8, 29, 3]. The research note that develops it [34] states its own boundary as: a finite, contractive, staged fragment of graded rho admits a candidate quantum interpretation, and the extension to unrestricted reflective concurrency remains open. That fence is where it is for good reasons, and a developer should read the complex row of the table above as “there is a real construction here for circuit-shaped programs” rather than “rholang is a quantum language”.

Learning under a moving target is not solved. The measured experiments include a regime shift, and under it every arm’s accuracy degrades in proportion to how much evidence it had accumulated. Accumulated confidence makes a learner rigid, and rigidity is exactly wrong when the world changes. This is a known gap, named as such in the companion note [35], and it is the most consequential one.

Two searches, not one. Grading gives a gradient *within* a hypothesis — tuning the numbers in a clause. It does not give a gradient *between* hypotheses — changing which formula the clause contains. That second search is discrete and looks like recombination rather than descent [40]. The split between the two is not an engineering decision; it follows from the shape of the hypothesis space.

6 Where this leaves a developer

6.1 The through-line in one page

The exposition began at a place every reader already knew and moved by necessity rather than by exposition. It is worth collecting the steps, because each was forced by the one before.

1. The comprehension `SELECT-FROM-WHERE` is what data access looks like, in every language that has one. It has no `DO`.
2. LINQ exists to embed it in a host language so that a developer can finally express the `DO`. Rholang goes the rest of the way and builds the language around the comprehension: `for ( ptrn <- chan where cond ) P`, with `chan! (Q)` as its dual.
3. Making both the read and the write explicit turns a communication into a meeting, and the substitution the rule performs is the witness. Transactions become a property of the program text: a failed guard is not rolled back because nothing was consumed.

4. That leaves the pattern and the condition unaccounted for, which forces a data definition language. MeTTaIL's `Theory` block supplies it in three rungs: grammar, equations, rewrites. The second rung holds all the invariants your type system cannot read; the third makes the data wiggle.
5. That structure subsumes MeTTa — rules are rewrites, atom spaces are channels — and adds the two things MeTTa lacks: an equational layer and a witnessed transaction.
6. The condition language is generated from the language definition rather than designed. Choosing the container for witnesses turns a dial between Boolean, linear and coalition sight, and the choice is an economic one about the deployment environment rather than a matter of rigour.
7. Grading the condition replaces the verdict with a value. At Booleans nothing changes; at the non-negative reals the program becomes a stochastic simulation of a population of itself; at the complex numbers, on a fenced fragment, a quantum one. Same text, two resolvers, no model drift.
8. In the graded setting the forager's guard is a learning rule, and the update is a correctly conditioned, unconditionally stable gradient step — arrived at without importing anything.

6.2 What to do on Monday

For a developer who wants to touch this rather than read about it, in order of increasing commitment:

- Run the two guard examples — the `receive` guard and the `match` fall-through — on the `cost-accounting-transpiler` branch of `flr3node-rust` [46]. They are small, they compile, and they are covered by tests. Change the guard and watch what does and does not get consumed.
- Run the cost-accounting demonstration in the same directory. It is a supply chain with conserved money and conserved inventory in which every operation is gated by a token, and it is the best available answer to what metering feels like when it is a program rather than a runtime flag.
- Write a `Theory` block for a data format you already own — ideally one whose type has an invariant currently maintained by a comment. Put the comment in the `equations` block. That single exercise is the fastest way to feel the difference between rung one and rung two.
- Then, if the shape of the thing appeals, add a `rewrites` block and discover that you have written a domain-specific language.

6.3 Where to read next

This note is deliberately the shallow end. The following are the deep ends, one line each.

Note	What it gives you that this does not
<i>Graph-Structured Lambda Theories: A Reference for the Working Developer</i> [30]	the same material with the theory first: what a language definition <i>is</i> , the category it lives in, and the two monads for cost and history
<i>Splitting the Mind of Zeus</i> [31]	why a logic has exactly the three components of §4, and where the container dial comes from
<i>Observation Disciplines</i> (omnibus) [32]	the full range of the dial worked through six disciplines, with measurements, and the argument that the disciplines compose but do not merge
<i>Graded Where-Clauses</i> [33]	the design and semantics of the graded slot, the resolution algebras, and the result that weight tables are a normal form of graded clauses
<i>Races Decided by the Born Rule</i> [34]	the complex-valued case in detail, including the fence around it
<i>Graded Where-Clauses over the Reals</i> [35]	the one-dimensional case: the gradient, the metric, and the measured forager
<i>The Mortal Scientist</i> [38]	the framework the running example is a small instance of — learning as cost-accounted computation

6.4 A closing observation

There is a temptation, reading a note like this, to treat the last two sections as the interesting part and the first two as preliminaries. Your author would put it the other way round.

The two widenings are widenings of one slot. That slot exists because the comprehension has a `WHERE` clause, and it has a `WHERE` clause because SQL had one in 1974 [5]. Nothing was invented to accommodate generated logics or graded values; a place for them was already present in the shape every developer already uses, and had been sitting empty. What this note describes is mostly the consequence of taking that shape seriously enough to build a language around it rather than embedding it in one.

The `DO` is the part LINQ went looking for. The rest of it is what you find when you keep going.

References

- [1] S. Aaronson. Quantum computing, postselection, and probabilistic polynomial-time. *Proceedings of the Royal Society A*, 461(2063):3473–3482, 2005.
- [2] S.-I. Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2):251–276, 1998.
- [3] C. Badescu and P. Panangaden. Quantum alternation: prospects and problems. In *Proceedings of Quantum Physics and Logic (QPL)*, EPTCS, 2015. arXiv:1511.01567.
- [4] L. Caires and L. Cardelli. A spatial logic for concurrency (part I). *Information and Computation*, 186(2):194–235, 2003.
- [5] D. D. Chamberlin and R. F. Boyce. SEQUEL: A structured English query language. In *Proceedings of the ACM SIGFIDET Workshop on Data Description, Access and Control*, pages 249–264, 1974.
- [6] E. F. Codd. A relational model of data for large shared data banks. *Communications of the ACM*, 13(6):377–387, 1970.
- [7] M. J. Gabbay and G. Losa. Semitopology: distributed collaborative action via topology, algebra and logic. Manuscript, 2023.

- [8] S. J. Gay and R. Nagarajan. Communicating quantum processes. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 145–157, 2005.
- [9] D. T. Gillespie. Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry*, 81(25):2340–2361, 1977.
- [10] J.-Y. Girard. Linear logic. *Theoretical Computer Science*, 50(1):1–101, 1987.
- [11] B. Goertzel et al. OpenCog Hyperon: a framework for AGI at the human level and beyond. Manuscript, SingularityNET, 2023.
- [12] B. Goertzel, M. Iklé, I. F. Goertzel, and A. Heljakka. *Probabilistic Logic Networks: A Comprehensive Framework for Uncertain Inference*. Springer, 2008.
- [13] M. Hennessy and R. Milner. Algebraic laws for nondeterminism and concurrency. *Journal of the ACM*, 32(1):137–161, 1985.
- [14] E. Meijer, B. Beckman, and G. Bierman. LINQ: reconciling object, relations and XML in the .NET framework. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, page 706, 2006.
- [15] R. Milner. Functions as processes. *Mathematical Structures in Computer Science*, 2(2):119–141, 1992.
- [16] G. D. Plotkin. A structural approach to operational semantics. Technical Report DAIMI FN-19, Aarhus University, 1981. Reprinted in *Journal of Logic and Algebraic Programming*, 60–61:17–139, 2004.
- [17] C. Priami. Stochastic π -calculus. *The Computer Journal*, 38(7):578–589, 1995.
- [18] A. Phillips and L. Cardelli. Efficient, correct simulation of biological processes in the stochastic π -calculus. In *Computational Methods in Systems Biology (CMSB)*, LNCS 4695, pages 184–199, 2007.
- [19] J. C. Reynolds. Separation logic: a logic for shared mutable data structures. In *Proceedings of the 17th IEEE Symposium on Logic in Computer Science (LICS)*, pages 55–74, 2002.
- [20] H. Robbins and S. Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [21] K. I. Rosenthal. *Quantales and Their Applications*. Pitman Research Notes in Mathematics 234. Longman, 1990.
- [22] M. Pauly. A modal logic for coalitional power in games. *Journal of Logic and Computation*, 12(1):149–166, 2002.
- [23] P. W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.
- [24] P. Wadler. List comprehensions. In S. L. Peyton Jones, *The Implementation of Functional Programming Languages*, chapter 7. Prentice Hall, 1987.
- [25] R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256, 1992.
- [26] S. Boag, D. Chamberlin, M. F. Fernández, D. Florescu, J. Robie, and J. Siméon. XQuery 1.0: An XML query language. W3C Recommendation, 2007.
- [27] J. Robie, D. Chamberlin, M. Dyck, D. Florescu, J. Melton, and J. Siméon. XQuery Update Facility 1.0. W3C Recommendation, 2011.
- [28] D. N. Yetter. Quantales and (noncommutative) linear logic. *The Journal of Symbolic Logic*, 55(1):41–64, 1990.

- [29] M. Ying, N. Yu, and Y. Feng. Alternation in quantum programming: from superposition of data to superposition of programs. *arXiv:1402.5172*, 2014.

F1R3FLY.io working notes. The following are in the `F1R3FLY-io/publications` repository; directory names are given so that they can be found.

- [30] L. G. Meredith. Graph-structured lambda theories: a reference for the working developer. F1R3FLY.io working note, 2026. `GSLT-intro/`.
- [31] L. G. Meredith. Splitting the mind of Zeus: packaging, splittings, and the three components of a logic. F1R3FLY.io working note, 2026. `on-the-nature-of-logic/`.
- [32] L. G. Meredith. Observation disciplines: how logics are generated from term structure, and why graded where-clauses are the mortal scientist’s instrument. F1R3FLY.io working note (omnibus), 2026. `rho-mind/observation-disciplines/`.
- [33] L. G. Meredith. Graded where-clauses: non-crisp truth values as the syntax of non-determinism resolution in the rho calculus. F1R3FLY.io working note, 2026. `fuzzyware/`.
- [34] L. G. Meredith. Races decided by the Born rule. F1R3FLY.io working note, 2026. `fuzzyware/`.
- [35] L. G. Meredith. Graded where-clauses over the reals: one-dimensional grading as the mortal scientist’s gradient, and probabilistic logic as the metric that gradient requires. F1R3FLY.io working note, 2026. `fuzzyware/`.
- [36] L. G. Meredith. Choice principles as graded where-clauses: one programming paradigm at every altitude of the Weihrauch lattice. F1R3FLY.io working note, 2026. `choice-types/`.
- [37] L. G. Meredith. Reading the herd: coalition logic as a graded hypothesis language for populations of computations. F1R3FLY.io working note, 2026. `consensus-types/`.
- [38] L. G. Meredith. The mortal scientist: learning as cost-accounted computation in a reflective higher-order calculus. F1R3FLY.io working note, 2026. `rho-life/`.
- [39] L. G. Meredith. Learning to play. F1R3FLY.io working note, 2026. `rho-life/`.
- [40] L. G. Meredith. Composing learners: populations as namespaces, parallel composition as the operator. F1R3FLY.io working note, 2026. `rho-life/`.
- [41] L. G. Meredith. Weighted graph-structured lambda theories: stochastic and quantum execution for MeTTaIL. F1R3FLY.io working note, second version, 2026. `rho-life/`.
- [42] M. Stay, L. G. Meredith, and J. B. Wells. Generating hypercubes of type systems. F1R3FLY.io working note, 2026. `drafts/Hypercube/`.
- [43] L. G. Meredith and M. Radestock. A reflective higher-order calculus. *Electronic Notes in Theoretical Computer Science*, 141(5):49–67, 2005.
- [44] L. G. Meredith and M. Radestock. Namespace logic: a logic for a reflective higher-order calculus. In *Trustworthy Global Computing*, LNCS 3705, pages 353–369, 2005.

Software.

- [45] F1R3FLY.io. `mettail-rust`: the language-definition macro and its generated toolchain. <https://github.com/F1R3FLY-io/mettail-rust>.
- [46] F1R3FLY.io. `f1r3node-rust`, branch `feature/cost-accounting-transpiler`: where guards, match guards, and the cost-accounting transpiler, with runnable examples under `examples/`. <https://github.com/F1R3FLY-io/f1r3node-rust>.